Exemple simple d'attaque CPA

Simon Abelard

November 2026

1 Objectifs du TP

Vous allez concevoir un environnement de simulation complet en Python pour :

1. Simuler les fuites physiques (consommation de courant) d'un microcontrôleur chiffrant une donnée avec l'algorithme AES-128.
2. Implémenter une attaque **CPA (Correlation Power Analysis)** basée sur le modèle du poids de Hamming pour retrouver le premier octet d'une clé secrète sans aucune connaissance préalable de celle-ci.

2 La cible : un XOR de l'AES

Lors du premier round de l'AES-128, le premier octet du texte clair P_0 subit une opération XOR ($\oplus$) avec le premier octet de la clé secrète K_0. Le résultat passe ensuite à travers une fonction de substitution non-linéaire appelée **S-Box**. La valeur intermédiaire sensible calculée par le processeur à cet instant précis est :

$$Y = \mathrm{SBox}(P_0 \oplus K_0) \tag{1}$$

Les circuits électroniques CMOS consomment de l'énergie principalement lorsque les portes logiques basculent d'un état à un autre. On modélise la consommation électrique instantanée de la puce comme étant proportionnelle au **Poids de Hamming** (noté HW, correspondant au nombre de bits à 1) de la valeur manipulée, noyée dans un bruit de fond de nature gaussienne.

3 Travail demandé

3.1 Partie 1 : Simulation des fuites

Dans cette première phase, vous devez générer vous-mêmes le jeu de données. Vous créerez une matrice de traces de dimension $N \times M$, où N est le nombre d'exécutions indépendantes (i.e. de traces) et M le nombre de points de mesure dans le temps.

1. Écrire une fonction `poids_de_hamming(valeur)` prenant en entrée un entier (ou un tableau d'entiers) et renvoyant son poids de Hamming.
2. Générer un échantillon de $N = 1000$ textes clairs aléatoires (valeurs comprises entre 0 et 255).
3. Choisir une clé secrète arbitraire K_0 (par exemple `0xAB`) et un instant temporel cible $T_{\text{fuite}} = 25$ (sur $M = 50$ points au total).
4. Construire la matrice des traces de consommation. Chaque trace doit initialement être composée uniquement d'un bruit blanc gaussien (d'écart-type $\sigma = 1.0$). Injecter ensuite la valeur théorique du poids de Hamming de Y à l'indice temporel T_{fuite}.

3.2 Partie 2 : Implémentation de l'attaque CPA

Vous basculez maintenant dans la peau de l'attaquant. Vous ne connaissez **que** la matrice de traces bruitées et le vecteur des textes clairs P_0. La clé secrète K_0 et l'instant T_{fuite} vous sont inconnus.

1. Pour chacune des 256 hypothèses de clé possibles ($K_{\text{sup}} \in [0, 255]$) :
   — Calculer le vecteur des états internes supposés : $Y_{\text{sup}} = \text{SBox}(P_0 \oplus K_{\text{sup}})$.
   — Calculer le vecteur de consommation théorique associé : $H_{\text{sup}} = HW(Y_{\text{sup}})$.
   — Pour chaque point temporel t allant de 0 à $M-1$, calculer le coefficient de corrélation linéaire de Pearson entre votre vecteur H_{sup} et la colonne t de la matrice des traces.
2. Extraire la valeur absolue maximale de corrélation obtenue pour chaque hypothèse. La clé maximisant cette métrique correspond à la clé secrète estimée.

4 Structure de code et S-Box de l'AES

```python
import numpy as np

# Table S-Box de l'AES-128
AES_SBOX = np.array([
    0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0x76,
    0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0xC0,
    0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0x15,
    0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0x75,
    0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0x84,
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A, 0x4C, 0x58, 0xCF,
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0xA8,
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0xD2,
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0x73,
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE, 0x5E, 0x0B, 0xDB,
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0x79,
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0x08,
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0x8A,
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0x9E,
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE, 0x55, 0x28, 0xDF,
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0x16
], dtype=np.uint8)

def poids_de_hamming(valeur):
    # A completer : Retourne le nombre de bits a 1
    pass
```

```
26
27 # --- PARTIE 1 : SIMULATION DES TRACES ---
28 N = 1000  # Nombre de traces
29 M = 50    # Nombre de points temporels
30 CLE_SECRETE = 0xAB
31 POINT_DE_FUITE = 25
32
33
34
35 # A completer : Generation des textes clairs et des traces bruitees
36 textes_clairs = np.random.randint(0, 256, N, dtype=np.uint8)
37 bruit = np.random.normal(0, 1.0, (N, M))
38 traces = bruit
39 traces = TODO
40
41 # --- PARTIE 2 : ATTAQUE CPA ---
42
43 #Pour chaque Ksup, calculer la consommation attendue
44
45 #Calcul du coefficient de Pearson
46     corr = np.corrcoef(consommation_theorique, traces[:, t])[0, 1]
47
48 #Recherche du maximum de correlation
```